PyTorch Notes: Building a Simple Neural Network (Single Neuron)

Introduction & Recap

In previous videos, you learned:

- **Introduction to PyTorch** – history, core features.
- **Tensors** – creation, operations.
- **Computation Graph** – how PyTorch tracks operations.
- **Autograd** – automatic differentiation.

Now we will build a **simple neural network with just one neuron** and train it on a **real-world dataset** (Breast Cancer dataset). The goal is to understand the **training pipeline** in PyTorch – even though we will write many parts manually, the structure is exactly what you'll use for complex networks later.

> **Note:** Our aim is **not** high accuracy, but learning the workflow.

Plan of Attack (Flowchart)

1. Load dataset (using pandas)
2. Preprocess data (scaling, encoding, train-test split)
3. Convert numpy arrays to PyTorch tensors
4. Define the neural network model (single neuron)
5. Set hyperparameters (learning rate, epochs)
6. Training loop (for each epoch):
   - Forward pass (compute predictions)
   - Calculate loss
   - Backward pass (compute gradients via `loss.backward()`)
   - Update parameters (weights and bias) using gradient descent
   - Zero gradients for next iteration
7. Evaluate model (accuracy on test set)

Step 1: Load and Preprocess Data

We use the **Breast Cancer dataset** (569 rows, 33 columns). Target column indicates cancer presence (binary classification). Unnecessary columns (`id`, `Unnamed: 32`) are dropped.

Code (Preprocessing – provided by instructor)

```
import pandas as pd
import numpy as np
```

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import torch

# Load data (assume df is loaded)
# Drop unnecessary columns
df.drop(['id', 'Unnamed: 32'], axis=1, inplace=True)

# Split features and target
X = df.drop('diagnosis', axis=1).values
y = df['diagnosis'].values

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Scale features (important for neural networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Encode labels (M = malignant, B = benign) to 1 and 0
encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)

# Convert numpy arrays to PyTorch tensors
X_train_tensor = torch.from_numpy(X_train).float()
X_test_tensor = torch.from_numpy(X_test).float()
y_train_tensor = torch.from_numpy(y_train).float()
y_test_tensor = torch.from_numpy(y_test).float()
```

Output Example

After conversion:

- `X_train_tensor` shape: `[455, 30]` (30 features)
- `y_train_tensor` shape: `[455]` with values 0 or 1

Step 2: Define the Neural Network (Single Neuron)

We create a class `MySimpleNeuralNetwork`. The neuron has:

- **30 weights** (one per input feature)
- **1 bias**

We initialize weights randomly, bias to zero. Both require gradients (`requires_grad=True`).

Code

```
class MySimpleNeuralNetwork:
    def __init__(self, X):
        # Number of features (30)
        self.weights = torch.rand(X.shape[1], 1, dtype=torch.float64,
requires_grad=True)
        self.bias = torch.zeros(1, dtype=torch.float64,
requires_grad=True)

    def forward(self, X):
        # Linear combination: z = X * w + b
        z = torch.matmul(X, self.weights) + self.bias
        # Sigmoid activation for binary classification
        y_pred = torch.sigmoid(z)
        return y_pred
```

Explanation

- `torch.rand(30,1)` creates a 30x1 matrix of random numbers.
- `torch.zeros(1)` creates a scalar bias initialized to 0.
- `requires_grad=True` enables gradient tracking for autograd.
- Forward pass: matrix multiply `X (n_samples × 30)` with `weights (30 × 1)` → `(n_samples × 1)`, add bias, apply sigmoid to get probabilities between 0 and 1.

Step 3: Hyperparameters

Set learning rate and number of epochs.

Code

```
learning_rate = 0.01
epochs = 25
```

Step 4: Create Model Instance

```
model = MySimpleNeuralNetwork(X_train_tensor)
```

Check initial weights and bias:

```
print(model.weights.shape)  # torch.Size([30, 1])
print(model.bias)            # tensor([0.], dtype=torch.float64,
requires_grad=True)
```

Step 5: Loss Function (Binary Cross-Entropy)

We manually implement binary cross-entropy loss:

`Loss = -[y * log(y_pred) + (1-y) * log(1-y_pred)]` averaged over all samples.

Code

```
def loss_fn(y_pred, y_true):
    # Add small epsilon to avoid log(0)
    epsilon = 1e-7
    y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
    loss = - (y_true * torch.log(y_pred) + (1 - y_true) * torch.log(1
- y_pred))
    return loss.mean()
```

Step 6: Training Pipeline

We loop over epochs and perform:

1. Forward pass → predictions
2. Compute loss
3. Backward pass → gradients (`loss.backward()`)
4. Update parameters using gradient descent (with `torch.no_grad()`)
5. Zero gradients for next epoch

Code

```
for epoch in range(epochs):
    # 1. Forward pass
    y_pred = model.forward(X_train_tensor)
    y_pred = y_pred.squeeze()  # shape [455] to match y_train_tensor

    # 2. Calculate loss
    loss = loss_fn(y_pred, y_train_tensor)

    # 3. Backward pass (compute gradients)
    loss.backward()

    # 4. Update parameters (without gradient tracking)
    with torch.no_grad():
        model.weights -= learning_rate * model.weights.grad
        model.bias -= learning_rate * model.bias.grad

    # 5. Zero gradients for next iteration
    model.weights.grad.zero_()
    model.bias.grad.zero_()
```

```
    # Print loss every epoch
    print(f'Epoch {epoch+1:2d}, Loss: {loss.item():.4f}')
```

Output Example (varies due to random initialization)

```
Epoch  1, Loss: 3.8762
Epoch  2, Loss: 2.5431
Epoch  3, Loss: 1.9873
...
Epoch 24, Loss: 0.5123
Epoch 25, Loss: 0.4987
```

Loss decreases over time, indicating learning.

Step 7: Evaluation (Accuracy on Test Set)

We compute predictions on test data, convert probabilities to class labels using a threshold (0.5), then calculate accuracy.

Code

```
# Predict on test set (no gradients needed)
with torch.no_grad():
    test_pred = model.forward(X_test_tensor).squeeze()
    # Convert probabilities to 0/1 using threshold 0.5
    test_pred_class = (test_pred > 0.5).float()

# Calculate accuracy manually
correct = (test_pred_class == y_test_tensor).sum().item()
total = y_test_tensor.shape[0]
accuracy = correct / total
print(f'Test Accuracy: {accuracy * 100:.2f}%')
```

Output Example

```
Test Accuracy: 51.75%
```

> Low accuracy is expected because we used a simple manual pipeline. In future videos, we will improve this using PyTorch's built-in modules, loss functions, and optimizers.

Summary of the Training Pipeline

Step	Operation	Code
1	Forward pass	`y_pred = model.forward(X)`

Step	Operation	Code
2	Compute loss	`loss = loss_fn(y_pred, y_true)`
3	Backward pass	`loss.backward()`
4	Update parameters	`with torch.no_grad(): param -= lr * param.grad`
5	Zero gradients	`param.grad.zero_()`

This is the **core framework** used in all PyTorch training loops.

What's Next? (Future Improvements)

In upcoming videos, we will replace manual components with PyTorch's built-in features:

Manual (current)	PyTorch built-in
Custom class with weights/bias	`torch.nn.Module` + `torch.nn.Linear`
Manual sigmoid	`torch.nn.Sigmoid()`
Custom loss function	`torch.nn.BCELoss()`
Manual gradient descent	`torch.optim.SGD` (or Adam, RMSprop)

The **pipeline structure remains the same** – only the internal implementations become more powerful and concise.

Key Takeaways

- PyTorch uses **tensors** with `requires_grad=True` to build computation graphs.
- `loss.backward()` computes gradients automatically.
- Always **zero gradients** after each update, otherwise they accumulate.
- Use `torch.no_grad()` when updating parameters or evaluating to avoid unnecessary gradient tracking.
- The training loop is consistent: forward → loss → backward → update → zero grad.

Now you are ready to write this pipeline in your copy and build upon it in future lessons!